

COMP90041

Programming and Software Development

Lecture 2 - Control Flows

Amani Abusafia

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Melbourne in accordance with section 113P of the *Copyright Act 1968 (Act)*.

The material in this communication may be subject to copyright under the Act.

Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice

Quick Recap

respond at [PollEv.com/abusafia](https://poll-ev.com/abusafia)



Control Flows – Overview



In this part of the lecture, you will learn about:

- Expressions, statements and blocks.
- Flow Controls
 - branching mechanisms
 - loops, and
- debugging.

Expressions, Statements and Blocks



Expression:

- **Expression** is a **combination** of variables, operators, and method invocations that evaluate to a **single** value.
- Expressions are **fundamental building blocks** for writing Java code.
- They are used in various contexts, such as:
 - Assigning values to variables
 - Performing calculations
 - Making decisions in conditional statements
 - Passing arguments to methods
 - Returning values from methods

Understanding how expressions work is crucial for writing correct and efficient Java code.

Expressions, Statements and Blocks



Expression:

Examples of expressions in Java:

- Arithmetic expression:
- Method invocation expression:
- Comparison expression:
- Logical expression:

`x + y * z`

`name.length()`

`age >= 18`

`isStudent && hasScholarship`

Expressions, Statements and Blocks



Statements:

- A **statement** in Java is a **complete instruction** that directs the program to perform a specific action.
- **Unlike expressions**, which always evaluate to a single value, statements don't necessarily produce a value but instead execute an action or change the program's state.

Expressions, Statements and Blocks



Statements:

key characteristics of statements in Java:

- **Types:** There are various **types of statements** in Java, each serving a specific purpose:
 - **Declaration:** Defining variables or methods.
 - e.g.: `int age = 25;      public void sayHello() {...}`
 - **Assignment:** Assigning values to variables
 - e.g.: `name = "John";`
 - **Control flow:** Statements controlling the program flow
 - e.g.: `if (x > 0) { ... }.      for (int i = 0; i < 10; i++) { ... }`
 - **Method invocation:** Calling functions within your code or external libraries
 - e.g.: `System.out.println("Hello");`
 - **Return:** Statements returning values from methods
 - e.g.: `return x;`

Expressions, Statements and Blocks



Statements:

key characteristics of statements in Java:

- **Execution:** Java code is executed line by line, meaning statements are **processed sequentially** according to their order in the program.
- **Scope:** Each statement exists within a specific scope, determining where it can be accessed and used.

Examples of statements in Java:

```
int x = 10;  
if (age > 18) {  
    System.out.println("You are eligible to vote.");  
}  
String greeting = sayHello("World");
```

Expressions, Statements and Blocks

Blocks:

- A block is a **group** of zero or more **statements** between balanced braces and can be used anywhere a single statement is allowed.
- The following example, illustrates the use of blocks:

```
class BlockDemo {                                // Start of Block 1 (class block)
    public static void main(String[] args) {      // Start of Block 2 (method block)
        boolean condition = true;

        if (condition) {                          // Start of Block 3 (if block)
            System.out.println("Condition is true.");
        }                                         // End of Block 3 (if block)

        else {                                    // Start of Block 4 (else block)
            System.out.println("Condition is false.");
        }                                         // End of Block 4 (else block)
    }                                             // End of Block 2 (method block)
}                                                 // End of Block 1 (class block)
```

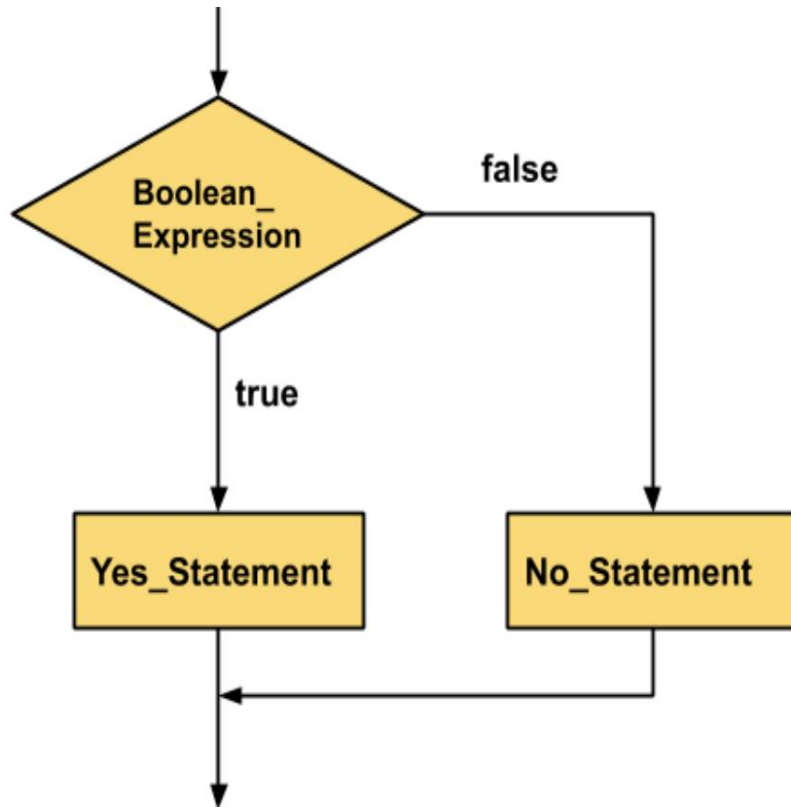
Control Flow



- As in most programming languages, **flow control** in Java refers to its **branching** and **looping** mechanisms.
- Java has several branching mechanisms:
 - if-else,
 - if,
 - and switch statements
- Most branching statements are controlled by **Boolean expressions**.
- A Boolean expression evaluates to either *true* or *false*.
- The primitive type **boolean** only takes the values *true* or *false*.

if-else Statements

- An if-else statement **chooses** between two alternative statements based on the value of a **Boolean expression**.



```
if (Boolean_Expression) {
```

```
    Yes_statement/s
```

```
}
```

```
else {
```

```
    No_statement/s
```

```
}
```

if-else Statements



- An if-else statement **chooses** between two alternative statements based on the value of a **Boolean expression**.

if (Boolean_Expression){

Yes_statement/s

}

else {

No_statement/s

}

single statement

```
if (condition)
    System.out.println("This is a single statement.");
```

compound statement

A compound statement is a list of statements enclosed by a pair of braces ({}).

```
if (condition) {
    System.out.println("This is the first statement in the compound statement.");
    // More statements can be added here
}
```

if-else Statements



Omitting the *else* Part

- The else part of an if-else statement **may be omitted** to obtain what is often only called an if statement.

```
int numberOfSales = 101;
int salary = 100000;
int bonus = 10000;

if (numberOfSales > 100) {
    salary += bonus;
    System.out.println("Congratulations! You received a bonus!");
}

System.out.println("Your salary this year: " + salary);
```

if-else Statements



```
int x = 5;  
if (x > 6)  
    System.out.println("Line 1.");  
    System.out.println("Line 2.");
```

The Output will be:

Line 2

if-else Statements



Nested Statements

- if-else and if statements can contain any single or compound statements, including other if-else and if statements.
- In this case, we speak of **nested statements**.
- Each level of a nested if-else or if should be indented further than the previous level to make your code

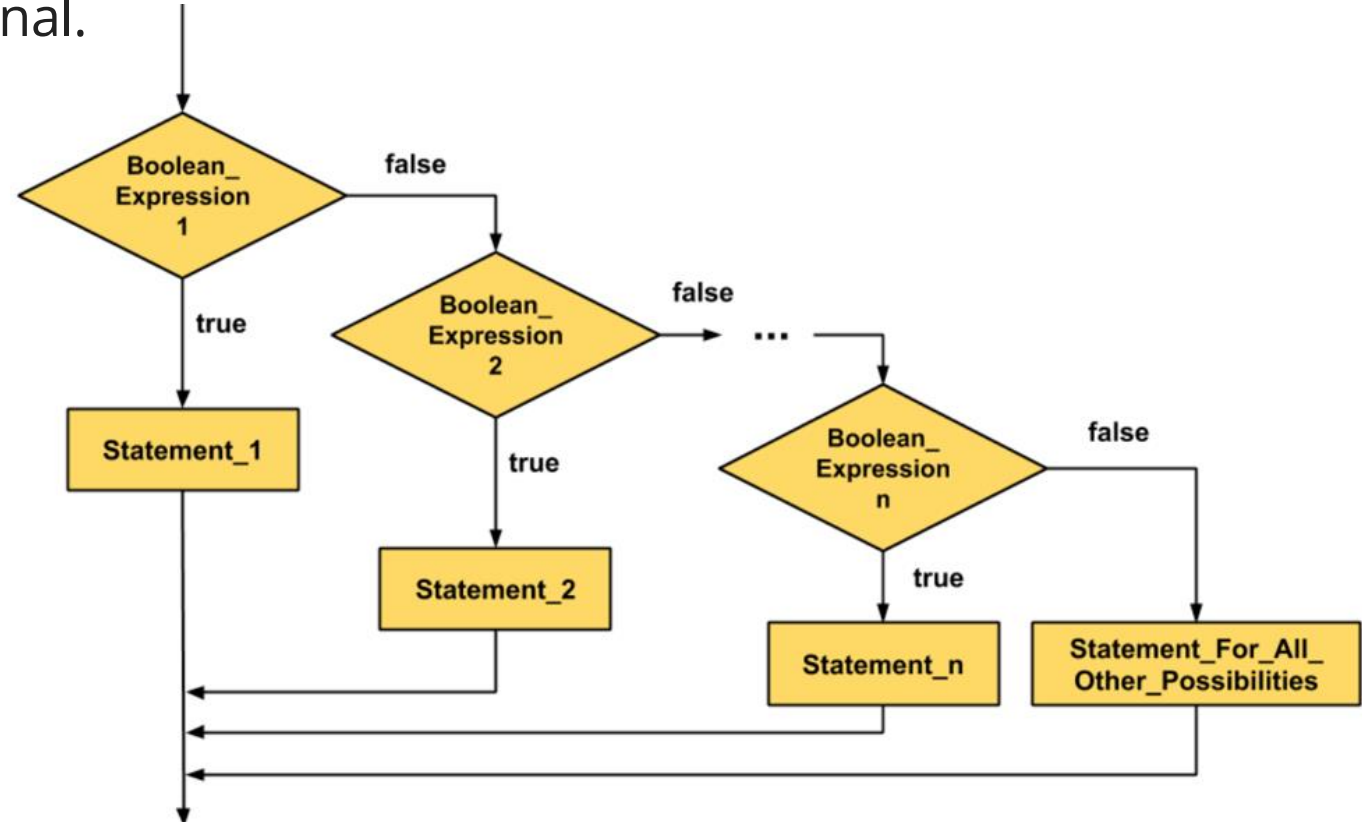
```
int income = 30000;  
int numberOfKids = 2;  
double tax = 0;
```

```
if (income > 18201) {  
    → tax = income * 0.19;  
    → if (numberOfKids>0) {  
        → tax = tax - 500; //tax relief  
    }  
}
```

if-else Statements

Multiway *if-else* Statements

- The multiway *if-else* statement is simply a normal *if-else* statement that **nests** another *if-else* statement at every *else* branch.
- The Boolean_Expressions are evaluated in the order they are written until one evaluates to *true*. The final *else* is optional.



if-else Statements

Multiway if-else Statements



```
int numberOfSales = 75;
int salary = 100000;
int smallBonus = 2000;
int mediumBonus = 5000;
int bigFatBonus = 10000;

if(numberOfSales > 200) {
    salary += bigFatBonus;
} else if(numberOfSales > 100) {
    salary += mediumBonus;
} else if(numberOfSales > 50) {
    salary += smallBonus;
} else {
    System.out.println("No bonus this year. :(");
}

System.out.println("Your salary this year: " + salary);
```

The Conditional Operator



- The **conditional operator** is a variant on certain forms of the if-else statement. It is also referred to as the **ternary operator** or **arithmetic if**:

```
if(n1>n2)
    max=n1;
else
    max = n2;
```

Using the ternary operator is equivalent to:

```
max = (n1 > n2) ? n1 : n2;
```

The *switch* Statement



- The **switch** statement is the only other kind of Java statement that implements **multiway** branching.
- When a *switch* statement is evaluated, one of different branches is executed.
- The choice of which branch to execute is determined by a controlling expression enclosed in parentheses after the keyword *switch*.

```
switch ( Controlling_Expression )  
{  
    case Case_Label_1:  
        Statement_Sequence_1  
        break;  
    case Case_Label_2:  
        Statement_Sequence_2  
        break;  
    case Case_Label_n:  
        Statement_Sequence_n  
        break;  
    default:  
        Statement_Sequence_n+1  
}
```

The *switch* Statement



- Case Labels:
 - Each case label must be of the **same type** as the controlling expression.
 - Case labels must be **unique**, i. e., may appear only once.
- Default Label:
 - The **optional default** label is usually used last. If no case label matches, then the only statements executed are those following the default label (if there is one).
 - It is a **good practice** to include a default section. In this case its not needed, it can, for example, be used to output an error message.

```
switch ( Controlling_Expression )  
{  
    case Case_Label_1:  
        Statement_Sequence_1  
        break;  
    case Case_Label_2:  
        Statement_Sequence_2  
        break;  
    case Case_Label_n:  
        Statement_Sequence_n  
        break;  
    default:  
        Statement_Sequence_n+1  
}
```

The *switch* Statement



- Each sequence of statements should be followed by a break statement, which tells the program execution to continue after the switch block.
- If you don't add a break statement, the next case(s) will be executed regardless of whether the case matches.
- The switch statement ends when it executes a break statement, or when the end of the switch statement is reached.

```
switch ( Controlling_Expression )  
{  
    case Case_Label_1:  
        Statement_Sequence_1  
        break;  
    case Case_Label_2:  
        Statement_Sequence_2  
        break;  
    case Case_Label_n:  
        Statement_Sequence_n  
        break;  
    default:  
  
}
```

The *switch* Statement



```
int dayOfWeek = 3;
```

```
switch (dayOfWeek) {  
    case 1:  
        System.out.println("Today is Monday");  
        break;  
    case 2:  
        System.out.println("Today is Tuesday");  
        break;  
  
    case 3:  
        System.out.println("Today is Wednesday");  
        break;  
}
```

The Output will be:

Today is Wednesday

The *switch* Statement



```
int dayOfWeek = 1;
```

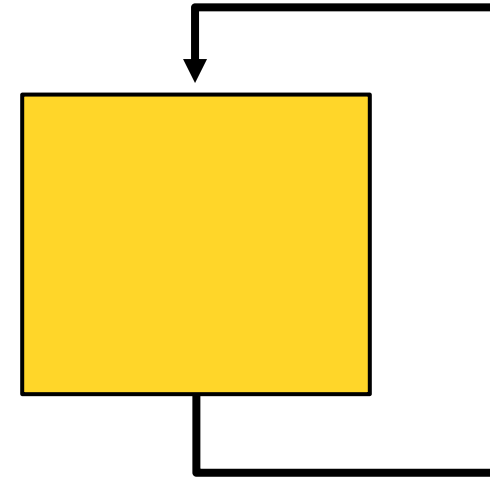
```
switch (dayOfWeek) {  
    case 1:  
    case 2:  
        System.out.println("Today is Tuesday");  
        break;  
  
    case 3:  
        System.out.println("Today is Wednesday");  
        break;  
}
```

The Output will be:

Today is Tuesday

Loops

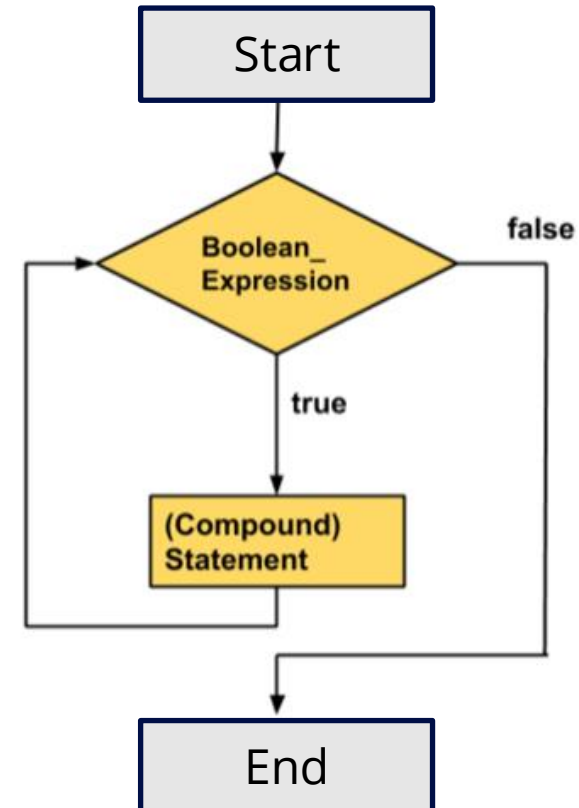
- **Loops** in Java are structures that allow you to **repeatedly** execute a block of code as long as a **specified condition** remains **true**.
- Java has three types of loop statements:
 - while
 - do-while,
 - and for statements
- The code that is repeated in a loop is called the **body** of the loop. Each repetition of the loop body is called an **iteration** of the loop.



The *while* Statement

```
while (Boolean_Expression)
{
    Compound_Statement
}
```

The `{ }` are required
when the loop body has
multiple statements



Beware of the dreaded **infinite loop** or **dead loop**

The while Statement

- Write a program to print 10 to 0

```
int count = 10;
```

```
while (count >= 0)
```

```
{
```

```
    System.out.println(count);
```

```
    count--;
```

```
}
```

Initialize the control variable

Validate the condition

Update the control variable

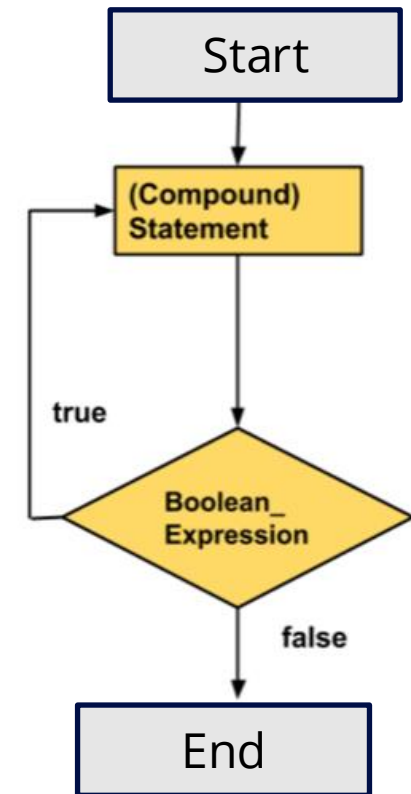
The *do while* Statement

A *do-while* statement executes the loop body **at least once**.

```
do  
{  
    Compound_Statement  
} while (Boolean_Expression) ;
```

The `{ }` are required
when the loop body has
multiple statements

The `;` is required after
the `Boolean_Expression`



The do while Statement

- Write a program to print from 10 to 0

```
int count = 10;
```

```
do
```

```
{
```

```
    System.out.println(count);
```

```
    count--;
```

```
} while (count >= 0);
```

Initialize the control variable

Update the control variable

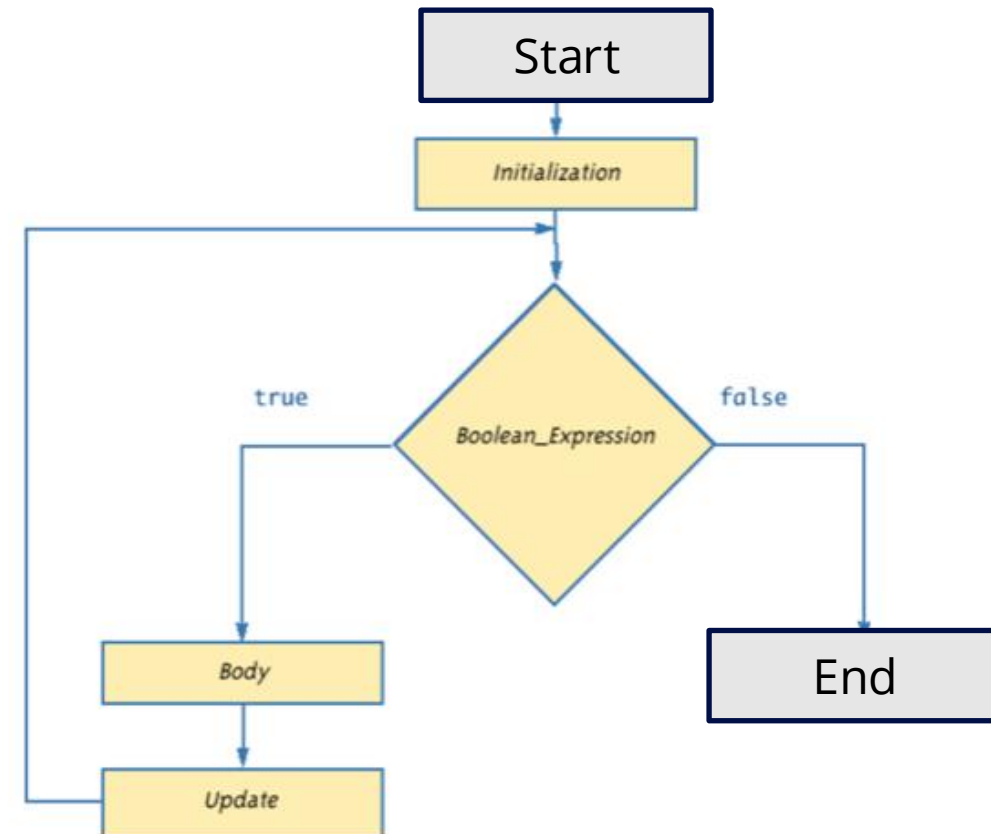
Validate the condition

The *for* Statement

```
for(Initializing; Boolean_Expression; Update)
{
    Compound_Statement
}
```

The `{ }` are required
when the loop body has
multiple statements

Beware the dreaded **infinite loop** or **dead loop**



The for Statement



Initialize the control variable

- Write a program to print 10 to 0

Validate the condition

```
for (int count = 10 ; count >= 0; count--)  
{  
    System.out.println(count);  
}
```

Update the control variable

Sneak-peak: for-each loops

There is another type of for loop that we will come across when we learn about data structures.

Nested Loops

- Loops can be **nested**, just like other Java structures.
- When nested, the **inner** loop iterates from beginning to end for every single iteration of the **outer** loop.

```
for (int row = 1 ; row < 4; row++)  
{  
    for (int col = 1 ; col < 5; col++)  
    {  
        System.out.print(row+"/"+col+"\t");  
    }  
    System.out.print("\n");  
}
```

Outer Loop

Inner Loop

The Output will be:

1/1	1/2	1/3	1/4
2/1	2/2	2/3	2/4
3/1	3/2	3/3	3/4

The *break* and *continue* Statements



- The *break* statement consists of the keyword break followed by a semicolon.
- It ends the nearest enclosing switch or loop statement.

```
Scanner scanner = new Scanner(System.in);  
while (true)  
{  
    System.out.print("Enter a number (0 to stop): ");  
    int num = scanner.nextInt();  
    if (num == 0)  
        break;  
    System.out.println("You entered: " + num);  
}
```

The *break* and *continue* Statements



- The **continue** statement consists of the keyword **continue** followed by a semicolon.
- When executed, the **continue** statement ends the current loop body iteration of the nearest enclosing loop statement.

```
Scanner scanner = new Scanner(System.in);  
int count = 0;  
while (count < 5)  
{  
    System.out.print("Enter a number :");  
    int num = scanner.nextInt();  
  
    if (num == 0)  
        continue;  
  
    System.out.println("You entered: " + num);  
    count++;  
}
```

Labelled break statement



- When loop statements are nested, any *break* or *continue* statement applies to the **innermost**, containing loop statement.
- **Labelled break statement** allows exiting any enclosing loop, not just the innermost one.
- Uses a **label (identifier)** to mark an outer loop.
- break label; **terminates the labelled loop** directly.
- Useful in **nested loops** to break out of multiple levels at once.

Labelled break statement

- Example

```
outer:    
for (int i = 1; i <= 3; i++)  
{  
    for (int j = 1; j <= 3; j++)  
    {  
        if (j == 2)  
            break outer;    
        System.out.println(i + ", " + j);  
    }  
}  
System.out.println("Loop exited.");
```

The *exit* Statement



- The *exit* method will immediately end the program as soon as it is invoked:

`System.exit(int status);`

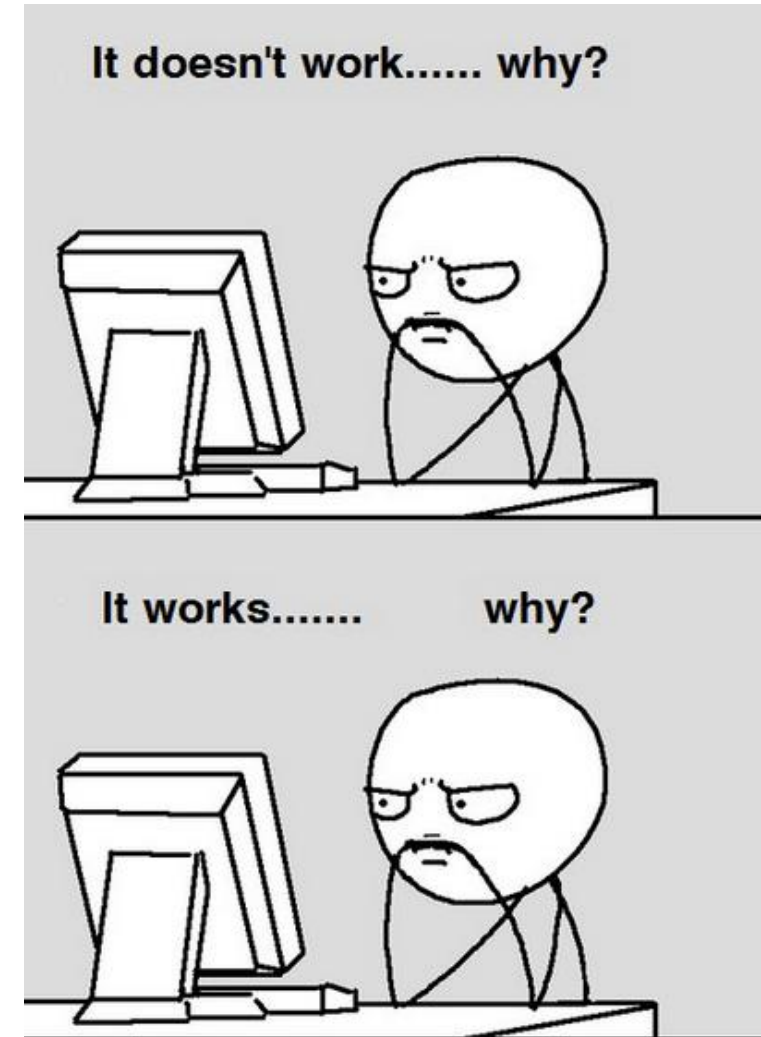
- The argument is an *integer* that indicates the exit status.
- `System.exit(0);` → Normal program termination.
- Non-zero values (e.g., `System.exit(1);`) → Indicate an error.
- Best practice:
 - Let the program reach the end of `main()` naturally.
 - Use `System.exit()` *only for errors* or critical stops.

Debugging



- A **bug** is an error or flaw in a program that leads to incorrect or unexpected program execution.
- The process of finding and fixing bugs is termed **debugging**.
- The following techniques will help you **inspect and understand your code** better:
 - **Tracing Variables:**
 - Watching one or more variables change value while a program is running.
 - Using Integrated Development Environments (IDEs)
 - OR use output statements

```
//tracing the variable count  
System.out.println("count=" + count);
```



Debugging



- The following techniques will help you **inspect and understand your code** better:
 - Using **Assertions**:
 - An **assertion** is used to **asserts** something about the state of a program
 - An assertion can be either true or false.
 - If false → Program **ends** and shows an ***assertion failed*** error message.
 - If true → Program **continues** normally.

assert Boolean_Expression;

- By default, assertions are off.
- Run with assertion checking enabled

```
$ java -enableassertions DrinkOrder
```

Debugging



- Example:

```
int age = 15;  
assert age >= 18 : "Age must be 18 or older!";  
System.out.println("Age is valid: " + age);
```



THE UNIVERSITY OF
MELBOURNE

Quick Practice

Quick Practice



True or False? The output will be **A**

```
int x = 5;  
if (x < 2);  
    System.out.println("A");
```

True



THE UNIVERSITY OF
MELBOURNE

Live Coding

Live Coding - Sum of Numbers



- Write a for loop that calculates the sum of the numbers from one to a hundred.
- Add an assertion to check the correctness of your result given that

$$sum = \frac{N * (N + 1)}{2}$$

Additional Reading Resources



- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 1, 2 and 3)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 4)
- Language Basics (accessible on 14-02-2024) Oracle's Java Documentation. Available at: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>

Break – 5 minutes

(c) The University of Melbourne 2023

This material is protected by copyright. Unless indicated otherwise, the University of Melbourne owns the copyright subsisting in the work. Other than for the purposes permitted under the Copyright Act 1968, you are prohibited from downloading, republishing, retransmitting, reproducing or otherwise using any of the materials included in the work as standalone files. Sharing University teaching materials with third-parties, including uploading lecture notes, slides or recordings to websites constitutes academic misconduct and may be subject to penalties. For more information: [Academic Integrity at the University of Melbourne](#)

Requests and enquiries concerning reproduction and rights should be addressed to the University Copyright Office, The University of Melbourne: copyright-office@unimelb.edu.au

See you next time!



THE UNIVERSITY OF
MELBOURNE